

אלגוריתמי דחיסה: אלגוריתם הופמן (Huffman)

אלגוריתם הופמן הוא אלגוריתם לדחיסת מידע ללא איבוד מידע. מטרת האלגוריתם היא להקטין את כמות המקום הדרושה לשמירת מידע, באמצעות שימוש בקודים קצרים יותר עבור תווים שמופיעים בתדירות גבוהה, וקודים ארוכים יותר עבור תווים שמופיעים בתדירות נמוכה. הרעיון המרכזי של אלגוריתם הופמן הרעיון המרכזי באלגוריתם הוא שלא כל התווים במידע מופיעים באותה תדירות. לדוגמה, בטקסט מסוים יכול להיות שיש אחד מופיע מאות פעמים, בעוד שיש אחר מופיע רק מספר פעמים. אם לכל התווים נשתמש באותו מספר ביטים, נבזבז מקום על תווים שמופיעים בתדירות גבוהה. אלגוריתם הופמן פותר את הבעיה הזאת באמצעות קוד באורך משתנה. לתווים שמופיעים הרבה פעמים ניתן קוד קצר, ולתווים שמופיעים מעט פעמים ניתן קוד ארוך יותר. כך ניתן להקטין את מספר הביטים הכולל הדרוש לייצוג המידע.

קוד הופמן

אלגוריתם הופמן משתמש בקודים בינאריים, כלומר קודים המורכבים מהספרות 0 ו 1. לכל תו מוקצה קוד בינארי ייחודי. הקודים אינם חייבים להיות באותו אורך. לדוגמה אם במידע מסוים התו A מופיע בתדירות גבוהה מאוד, ניתן לתת לו קוד קצר כמו 0. לעומת זאת לתו שמופיע לעיתים רחוקות ניתן קוד ארוך יותר, לדוגמה 1101. המשמעות היא שהתווים הנפוצים מיוצגים באמצעות פחות ביטים, ולכן ניתן לחסוך במקום. כדי ליצור את הקודים, האלגוריתם מתחיל בספירת מספר הפעמים שכל תו מופיע במידע. לאחר מכן יוצרים רשימה של כל התווים יחד עם התדירות שלהם. האלגוריתם מחפש בכל פעם את שני התווים או הצמתים בעלי התדירות הנמוכה ביותר ומאחד אותם לצומת חדש. התדירות של הצומת החדש היא הסכום של שתי התדירויות שחוברו. התהליך ממשיך עד שכל התווים מחוברים לעץ אחד שנקרא Huffman Tree. לאחר בניית העץ, ניתן לעבור מהשורש אל כל תו וליצור עבורו קוד בינארי. בדרך כלל מעבר לצד אחד בעץ מסומן ב-0 ומעבר לצד השני מסומן ב-1. כך לכל תו מתקבל קוד ייחודי.

עץ הופמן

עץ הופמן הוא מבנה הנתונים שבו משתמש האלגוריתם כדי ליצור את הקודים הבינאריים. בתחתית העץ נמצאים התווים עצמם, ובכל צומת נשמרת תדירות הופעת התווים שנמצאים מתחתיו. כאשר עוברים מהשורש לכיוון תו מסוים, אוספים את הביטים שהתקבלו בדרך. רצף הביטים הזה הוא הקוד של אותו תו. תווים שמופיעים בתדירות גבוהה ממוקמים בדרך כלל קרוב יותר לשורש ולכן מקבלים קוד קצר יותר. תווים שמופיעים בתדירות נמוכה נמצאים בדרך כלל עמוק יותר בעץ ולכן מקבלים קוד ארוך יותר. דוגמה פשוטה

נניח שיש לנו את המידע הבא:

AAAAABBC

התדירות של התווים היא:

A מופיע 5 פעמים.

B מופיע 2 פעמים.

C מופיע פעם אחת.

מכיוון שהתו A מופיע בתדירות הגבוהה ביותר, האלגוריתם ישאף לתת לו קוד קצר יותר. לעומת זאת, התו C מופיע פעם אחת בלבד ולכן יקבל קוד ארוך יותר.

לדוגמה, קוד אפשרי יכול להיות:

A = 0

B = 10

C = 11

במקום להשתמש באותו מספר ביטים עבור כל תו, התו הנפוץ A מיוצג באמצעות ביט אחד בלבד. כאשר מקודדים את המידע באמצעות הקודים האלה, מספר הביטים הכולל יכול להיות קטן יותר לעומת שיטה שבה לכל תו יש קוד באורך קבוע. חשוב להבין שהקודים בדוגמה הם רק דוגמה. הקודים המדויקים תלויים בתדירויות של כל התווים ובמבנה עץ הופמן שנוצר.

תכונה חשובה: Prefix Code

אחת התכונות החשובות של קוד הופמן היא שאף קוד של תו אחד אינו מהווה התחלה של קוד של תו אחר. תכונה זו נקראת Prefix Free Code.

לדוגמה, אם לתו מסוים יש את הקוד 0, לא ניתן לתת לתו אחר קוד שמתחיל ב-0. הדבר חשוב מכיוון שהוא מאפשר למערכת להבין היכן מסתיים קוד של תו אחד ומתחיל הקוד של התו הבא, גם כאשר אורך הקודים שונה. בזכות תכונה זו ניתן לפענח את המידע ללא צורך בסימני הפרדה בין התווים.

פתיחת הדחיסה

כדי לפתוח מידע שנדחס באמצעות הופמן, המקבל צריך לדעת את עץ הופמן או מידע אחר שמאפשר לו לבנות מחדש את הקודים.

הפענוח מתבצע באמצעות מעבר בעץ לפי הביטים שהתקבלו.

כאשר מתקבל 0 עוברים לכיוון אחד בעץ, וכאשר מתקבל 1 עוברים לכיוון השני.

כאשר מגיעים לתו בעץ, יודעים שזהו התו הבא במידע המקורי וחוזרים לשורש כדי לפענח את הביטים הבאים.

בגלל שהקודים הם Prefix Free, ניתן לפענח את המידע בצורה חד משמעית.

למה הופמן חוסך מקום?

החיסכון של אלגוריתם הופמן נובע מכך שהתווים הנפוצים מקבלים קודים קצרים יותר.

אם תו מסוים מופיע מספר גדול של פעמים, אפילו חיסכון של ביט אחד בכל הופעה שלו יכול ליצור חיסכון משמעותי בגודל המידע הכולל.

לעומת זאת, תווים נדירים מקבלים קודים ארוכים יותר, אך מכיוון שהם מופיעים מעט פעמים, ההשפעה שלהם על הגודל הכולל קטנה יותר.

לכן השיטה מתאימה במיוחד למידע שבו יש הבדל משמעותי בתדירות ההופעה של התווים.

דחיסה ללא איבוד מידע

אלגוריתם הופמן הוא אלגוריתם דחיסה ללא איבוד מידע.

המשמעות היא שהמידע לאחר פתיחת הדחיסה זהה לחלוטין למידע המקורי.

היתרון המרכזי של אלגוריתם הופמן הוא האפשרות להקטין את גודל המידע ללא איבוד נתונים.

אלגוריתם הופמן נמצא בשימוש במגוון שיטות דחיסה והוא משמש גם כחלק מאלגוריתמי דחיסה מורכבים יותר.

אחת הדוגמאות המוכרות היא DEFLATE, כמו אצלנו בפרויקט שיטה שמשלבת רעיונות של LZ77 יחד עם קידוד הופמן.

השילוב בין השיטות מאפשר קודם לזהות רצפים שחוזרים על עצמם ולאחר מכן לייצג את המידע שהתקבל בצורה יעילה יותר באמצעות קודי הופמן.

לכן LZ77 והופמן יכולים לעבוד יחד ולספק דחיסה יעילה יותר מאשר שימוש בכל אחת מהשיטות בנפרד. LZ77 והופמן הם שני אלגוריתמים שונים, אך שניהם משמשים לדחיסה ללא איבוד מידע.

LZ77 מתמקד בזיהוי רצפים שחוזרים על עצמם. במקום לשמור את אותו רצף מספר פעמים, הוא משתמש בהפניה לרצף שכבר הופיע.

לעומת זאת, הופמן מתמקד בתדירות ההופעה של תווים. תווים שמופיעים הרבה מקבלים קודים קצרים, ותווים שמופיעים מעט מקבלים קודים ארוכים יותר.

לכל שיטה יש יתרונות משלה, אך ניתן לשלב ביניהן. בשיטות דחיסה מסוימות משתמשים קודם ברעיון של LZ77 כדי לזהות חזרות ולאחר מכן בקידוד הופמן כדי לייצג את התוצאה בצורה יעילה יותר.